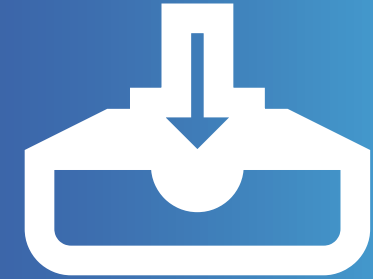
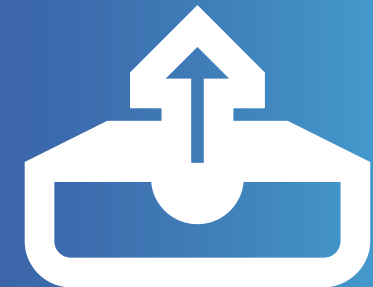

Feature Launch Kit

Release notes in. A full launch kit out.

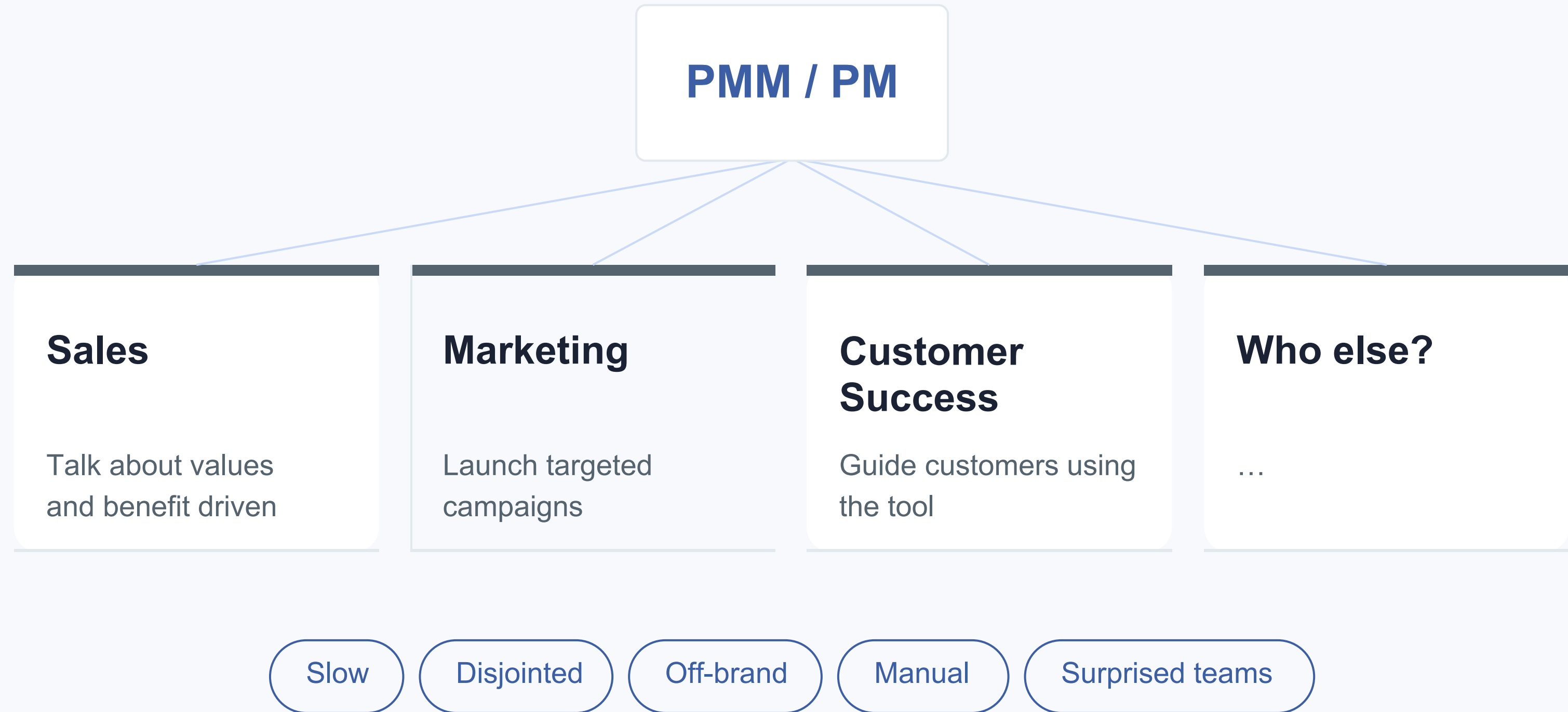


RELEASE NOTES IN

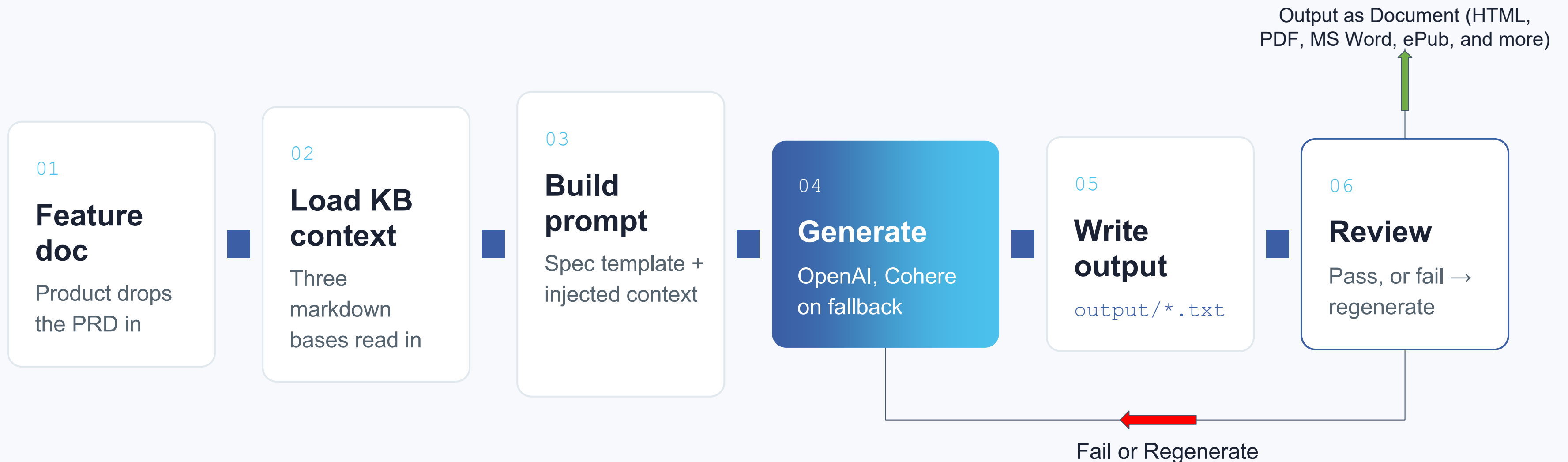


LAUNCH ASSETS OUT

One release, many dependencies, many assets



The generation pipeline



— The review gate is a person. A failed asset loops back to step 3 with a sharpened spec.

~\$0.008

per full launch kit — all three outputs

500+

full runs inside the €5 budget

3

documents per run, one API call each

Built to run cheap

Enterprise output, hobby-project budget.

- **Cheap by default**

gpt-4o-mini for generation; gpt-4o only for final showcase outputs. One config line, no code change.

- **Model lives in .env**

Not hardcoded — trade quality against cost per run without touching the pipeline.

- **Fallback is not double spend**

Cohere only fires when OpenAI fails, so resilience costs nothing while things work.

- **Context injection, not RAG**

No vector database, no embedding calls, no infrastructure bill.

Model tier is a dial, not a rewrite — the same pipeline scales from free-tier testing to production-grade output.

What was hard, and what we take forward

Cross-provider fallback

Two APIs with different response shapes behind one interface took more care than the prompt work

Prompts non-devs can edit

Keeping specs in markdown meant rewriting the builder to be generic instead of per-asset

Scoping RAG out

Defending a "no" with criteria was harder, and more useful, than building it

WHERE THIS GOES

From one feature to every release.

Why we did not use RAG

Context injection

01

Corpus size

A small, curated set of markdown files that are linked directly to a single prompt. Nothing to retrieve from for MVP.

02

Query pattern

Three fixed outputs directly from given sources — not an open-ended search across a library.

03

Complexity vs. scope

Adding a vector store and a chunking strategy are much more complex given the small scope.

We revisit RAG when the corpus does: many features across a release database, where retrieval beats stuffing the prompt.

Three features we would build next

01

Support macro pack

A fourth output: ticket macros and FAQ entries for Customer Success, generated from the same feature doc.

NEXT STEP

Write the markdown spec and run it against last release's PRD

02

Review feedback loop

Capture why a reviewer rejected failed an output asset and feed those notes back into the LLM spec for iterative improvement.

NEXT STEP

Log fail reasons to a review file the builder reads on the next run

03

Individual release mode

Run individual feature in a release in one pass — To create independent modes of output that fit with the needs of the respective department. (PDF, Email, Slides etc.)

NEXT STEP

Measure cost per feature, then re-open the RAG decision

All three fit the current architecture — new specs and orchestration, no rewrite.

Three defences against generic output

Contextual grounding

Claims come from our KBs — real persona pain points, real product names, real brand voice

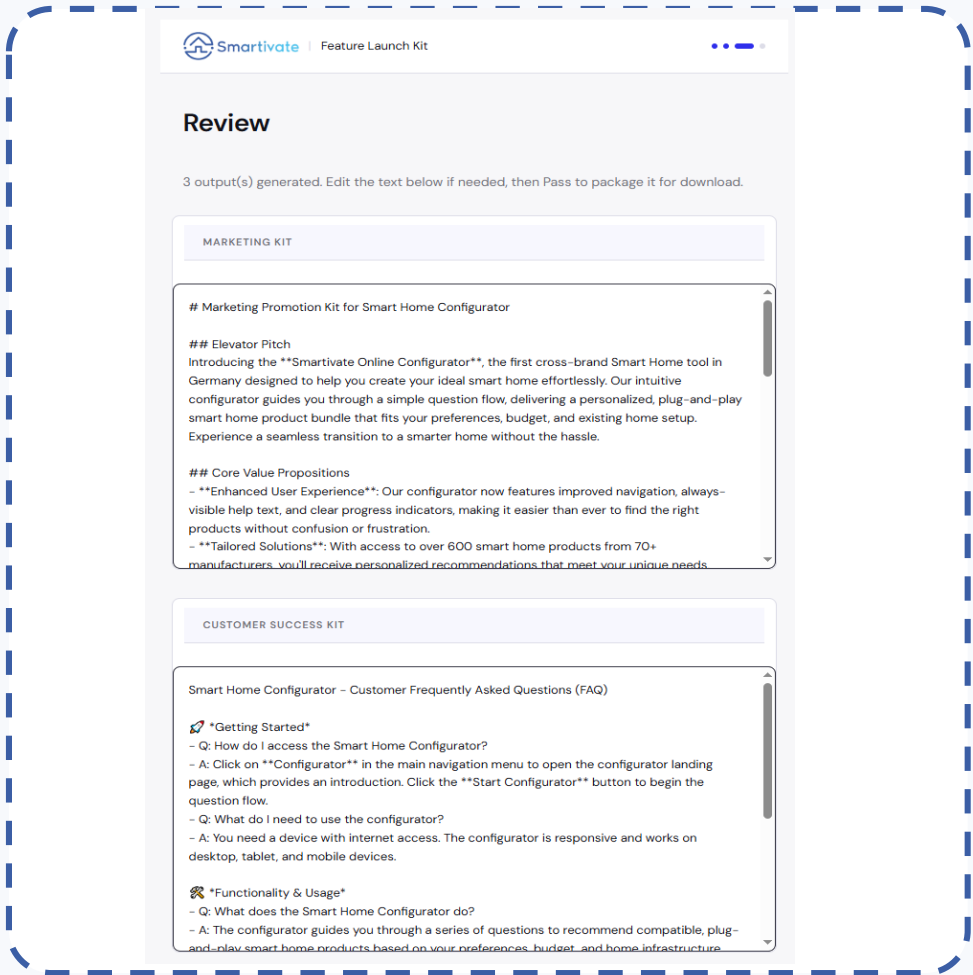
Human in the loop

Every asset passes a review gate before it counts as done; failures regenerate

Style discipline

Prompt rules ban buzzwords like "game-changer" and "seamless", and force claims to trace to a source doc

Feature
Launch
Kit



ChatGPT

Smart Home Configurator – Communications Pack

[Marketing Email](#) [Sales One Pager](#) [User FAQ](#) [Source PRD](#)

1. Marketing Email

Subject: Discover a Faster, Easier Smart Home Configuration Experience

We're excited to introduce an improved Smart Home Configurator designed to help customers find the right plug-and-play smart home bundle with greater confidence. The experience now features clearer navigation, always-visible explanation of better progress tracking, and improved visual feedback when making selections.

- Navigate without unnecessary scrolling.
- See helpful explanations without clicking.
- Track exactly where you are in the journey.
- Feel confident your selections have been saved.

Try the new configurator today and discover your ideal smart home solution in just a few minutes.

2. Sales One Pager

Core Value

The enhanced Smart Home Configurator makes it easier for customers to select the right products, reducing confusion and increasing confidence.

Customer Benefits

- Faster decision making.
- Greater transparency throughout the buying journey.
- Improved understanding of product categories.
- Consistent and intuitive user experience.

Business Benefits

Thank you

Questions?

Provider fallback

The RAG decision

Prompt specs

Cost per run